Universidad de Córdoba (UCO)

Ingeniería Informática

Asignatura:
Ingeniería de Sistemas Software Basados en Conocimiento

Trabajo Final de la Asignatura

Sistema Experto de Diagnóstico Nutricional (Examen 17 de Marzo)

Alumno: Álvaro Jurado Pozo
Usuario UCO: i32jupoa
Profesor: Antonio Calvo Cuenca

18 de marzo de 2026

Índice

1. Introducción **2**

2. Descripción de la aplicación **2**

3. Enlaces y repositorios del proyecto **2**
3.1. Código desarrollado (Ficheros en www-docs) 3
3.2. Repositorio en GitHub 3
3.3. Vídeo explicativo en YouTube 4

4. Estructura del Proyecto **4**

5. Desarrollo del código: Arranque y Vista **5**
5.1. Punto de entrada de la aplicación 5
5.2. Diseño de la ventana principal 5
5.3. Ventanas de diálogo secundarias 7

6. Desarrollo del código: Controlador de Eventos **9**

7. Conclusiones **10**

1. Introducción

El objetivo fundamental ha sido aplicar los conceptos de programación y arquitectura de software adquiridos durante el curso para diseñar, desarrollar y desplegar un sistema experto completamente funcional.

Este sistema se orienta al ámbito de la salud, específicamente al pre-diagnóstico nutricional automatizado, demostrando la viabilidad de integrar interfaces gráficas de usuario modernas, motores de inferencia clásicos basados en reglas estáticas y modelos de inteligencia artificial generativa de ejecución local. A lo largo de este documento, detallaré mi contribución específica a este desarrollo colaborativo, centrada en la capa de presentación, el control de eventos y la lógica del sistema de respaldo.

2. Descripción de la aplicación

El presente documento detalla el desarrollo de un Sistema Experto de Diagnóstico Nutricional, elaborado como trabajo final de la asignatura y evaluado en la prueba práctica del 17 de marzo. La aplicación ha sido construida utilizando el lenguaje de programación Python y la biblioteca gráfica PyQt6. Todo el proyecto se ha estructurado siguiendo el patrón de arquitectura Modelo-Vista-Controlador, lo cual permite una separación clara de responsabilidades y facilita el mantenimiento del código a largo plazo.

El propósito principal de este software es brindar una herramienta de apoyo clínico para la realización de pre-diagnósticos nutricionales automatizados. Para ello, el sistema recopila y analiza tres bloques de datos fundamentales introducidos por el usuario: la sintomatología física del paciente, sus parámetros biométricos como el índice de masa corporal y las horas de descanso diario, y la frecuencia semanal de consumo de diversos grupos alimenticios.

Dentro de la división de tareas del equipo, mi rol principal ha abarcado la gestión inicial del control de versiones mediante la creación del repositorio en GitHub, así como la producción y subida del vídeo demostrativo a YouTube. A nivel de programación, me he encargado del desarrollo integral de la capa de vista, diseñando tanto la ventana principal como los cuadros de diálogo secundarios. He implementado también el controlador principal que orquesta los eventos y comunica la vista con el modelo.

Por otro lado, he programado el motor de inferencia local de respaldo. Este componente actúa como un sistema basado en reglas clínicas estrictas que opera sin necesidad de conexión a internet ni inteligencia artificial, garantizando que la aplicación pueda emitir un informe estructurado de forma instantánea mediante el cruce de las variables dietéticas y sintomatológicas.

3. Enlaces y repositorios del proyecto

En esta sección se facilitan los enlaces requeridos para la evaluación de la prueba, incluyendo el acceso detallado al código fuente alojado en el servidor del departamento, el repositorio de control de versiones y el material audiovisual explicativo.

3.1. Código desarrollado (Ficheros en www-docs)

Los archivos ejecutables y módulos descritos en la arquitectura del software se encuentran subidos al servidor de la universidad para su comprobación directa. A continuación, se detalla el enlace a cada uno de los ficheros que componen la aplicación:

- **Ejecutable Principal:** https://www.uco.es/~i32jupoa/examen_final/main.py
- **Modelo:** https://www.uco.es/~i32jupoa/examen_final/modelo/nutricion_model.py
- **Controlador:** https://www.uco.es/~i32jupoa/examen_final/controlador/main_controller.py
- **Vista (Principal):** https://www.uco.es/~i32jupoa/examen_final/vista/principal_view.py
- **Vista (Diálogos):** https://www.uco.es/~i32jupoa/examen_final/vista/dialogos_view.py

3.2. Repositorio en GitHub

Para el desarrollo colaborativo y el control de versiones, procedí a crear un repositorio en la plataforma GitHub al inicio del proyecto. En él se han ido integrando las distintas ramas de desarrollo correspondientes a la vista, el modelo y el controlador. El acceso al código fuente se puede realizar desde el siguiente enlace:
https://github.com/i32jupoa/Trabajo_fin_curso.git

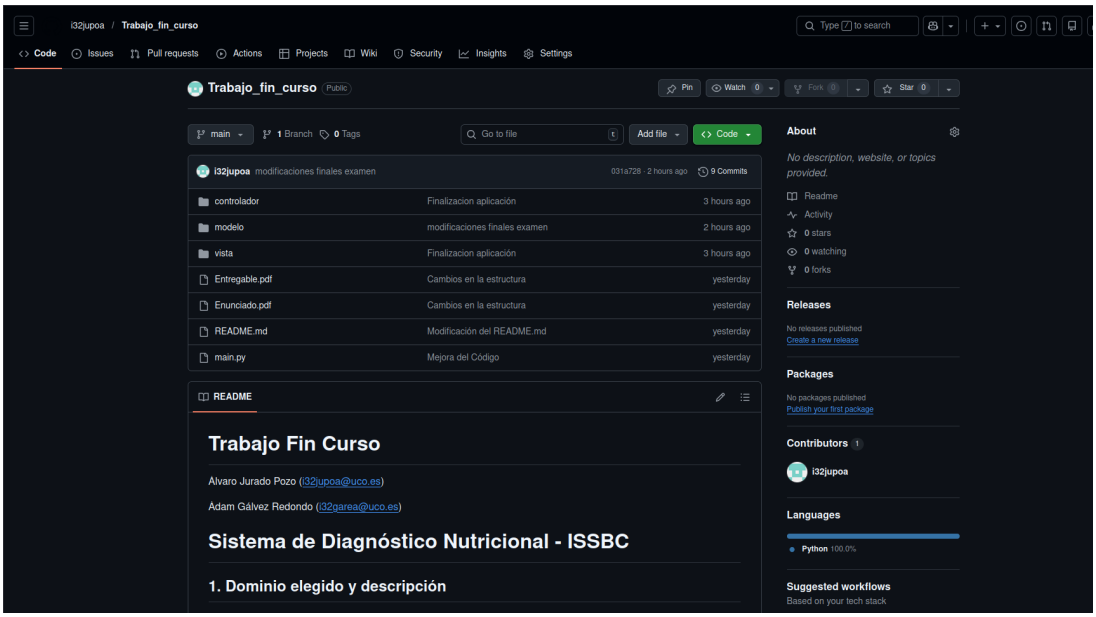

Figura 1: Captura de pantalla del repositorio de GitHub gestionado para el proyecto.

3.3. Vídeo explicativo en YouTube

Como parte de los entregables, me he encargado de grabar y editar un vídeo demostrativo utilizando la herramienta OBS Studio. En este material audiovisual explicamos la funcionalidad de la aplicación en tiempo real, el flujo de ejecución de los diagnósticos y los detalles de la implementación arquitectónica. Posteriormente, he subido el vídeo a la plataforma YouTube, el cual puede visualizarse en el siguiente enlace:

https://youtu.be/muXQWYvSjtg?is=aocPOYAlixrjMvez

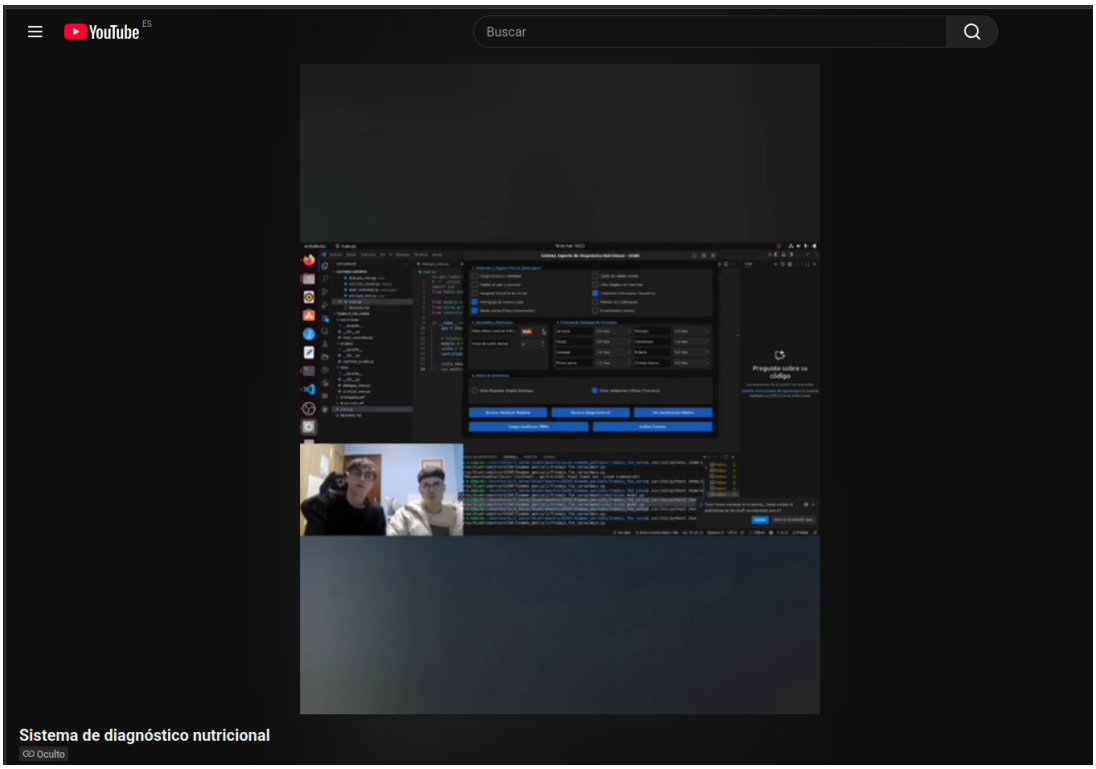

Figura 2: Captura de pantalla del vídeo demostrativo subido a la plataforma YouTube.

4. Estructura del Proyecto

Para mantener un orden riguroso y cumplir con el patrón Modelo-Vista-Controlador, he organizado el espacio de trabajo en diferentes directorios y ficheros. La jerarquía del proyecto, tal y como se refleja en el entorno de desarrollo y en los enlaces facilitados anteriormente, es la siguiente:

- **Directorio raíz:** Contiene el archivo de arranque de la aplicación. Se encarga de instanciar el modelo, la vista y el controlador, y de iniciar el bucle de eventos de la interfaz gráfica.
- **Directorio controlador:** Contiene el archivo donde he programado toda la lógica de conexión entre los botones de la vista y las funciones del modelo.
- **Directorio modelo:** Carpeta que alberga el fichero con la lógica de negocio, las reglas del motor de inferencia local y la conexión con la inteligencia artificial.

- **Directorio vista:** Responsable de la interfaz gráfica. Contiene el diseño de la ventana principal de la aplicación y la implementación de las ventanas emergentes para mostrar resultados y gestionar archivos.

5. Desarrollo del código: Arranque y Vista

En este apartado detallo la implementación del punto de entrada de la aplicación y el diseño completo de la interfaz de usuario, justificando las decisiones técnicas tomadas para construir una experiencia de uso fluida y estructurada.

5.1. Punto de entrada de la aplicación

El fichero principal de la aplicación actúa como el núcleo orquestador del patrón de arquitectura utilizado. Su única responsabilidad es inicializar el bucle de eventos de la biblioteca gráfica y ensamblar los tres componentes principales del sistema. Primero instancio el modelo de datos, luego la vista principal, y finalmente le paso ambas instancias al controlador para que establezca las conexiones necesarias antes de mostrar la ventana en pantalla.

```python
# -*- coding: utf-8 -*-
import sys
from PyQt6.QtWidgets import QApplication

from modelo.nutricion_model import ModeloNutricion
from vista.principal_view import VistaPrincipal
from controlador.main_controller import ControladorNutricion

if __name__ == '__main__':
    app = QApplication(sys.argv)

    # Ensamblaje MVC
    modelo = ModeloNutricion()
    vista = VistaPrincipal()
    controlador = ControladorNutricion(modelo, vista)

    vista.show()
    sys.exit(app.exec())
```

Imagen 1: Código del fichero de arranque main.py

5.2. Diseño de la ventana principal

El archivo correspondiente a la vista principal contiene la clase que hereda de la ventana principal de Qt. En su constructor, establezco las propiedades fundamentales como el título y un tamaño mínimo para asegurar que todos los controles sean visibles. Además, hago una llamada al método encargado de inyectar las hojas de estilo en cascada.

He diseñado un tema visual de noche profunda orientado a reducir la fatiga ocular en entornos clínicos. Para lograr esto, he modificado los colores de fondo hacia tonos grises muy oscuros y he ajustado el color del texto para garantizar un alto contraste. He prestado

especial atención a forzar el dibujo de las casillas de verificación mediante código para asegurar su correcta visualización independientemente del sistema operativo utilizado.

```
    def aplicar_estilos(self):
        self.setStyleSheet("""
            QWidget {
                background-color: #121212;
                color: #cccccc;
                font-family: Arial;
                font-size: 13px;
            }
            QGroupBox {
                background-color: #1e1e1e;
                border: 1px solid #333333;
                margin-top: 15px;
                border-radius: 6px;
            }
            /* Los elementos interactivos reciben colores de resalte en
               tonos azules */
            QCheckBox::indicator:checked {
                background-color: #155bb5;
                border: 2px solid #155bb5;
            }
        """)
```

Imagen 2: Aplicación de estilos visuales en principal_view.py

La disposición de los elementos en la ventana se organiza mediante contenedores de agrupamiento. El primer bloque se dedica a la recolección de los síntomas físicos del paciente. Para distribuir las diez casillas de verificación de manera uniforme, he implementado un diseño de cuadrícula que divide las opciones en dos columnas.

El segundo gran bloque recoge las variables biométricas y los hábitos dietéticos. He configurado controles numéricos incrementables para el peso y las horas de descanso, asegurando que los botones de incremento y decremento sean siempre visibles. Para la dieta, he dispuesto múltiples menús desplegables que recogen la frecuencia semanal de consumo de alimentos estructurados también en un formato de cuadrícula.

```
        grupo_observables = QGroupBox("2. Biometr a y Descanso")
        layout_observables = QFormLayout()

        self.spin_imc = QDoubleSpinBox()
        self.spin_imc.setRange(10.0, 50.0)
        self.spin_imc.setValue(22.0)
        self.spin_imc.setButtonSymbols(QAbstractSpinBox.ButtonSymbols.
            PlusMinus)

        self.spin_sueno = QSpinBox()
        self.spin_sueno.setRange(0, 24)
        self.spin_sueno.setValue(7)
        self.spin_sueno.setButtonSymbols(QAbstractSpinBox.ButtonSymbols.
            PlusMinus)

        layout_observables.addRow(" ndice  Masa Corporal (IMC):", self.
            spin_imc)
        layout_observables.addRow("Horas de sue o diarias:", self.
            spin_sueno)
```

```
        grupo_observables.setLayout(layout_observables)
```

Imagen 3: Implementación de los controles biométricos y la dieta

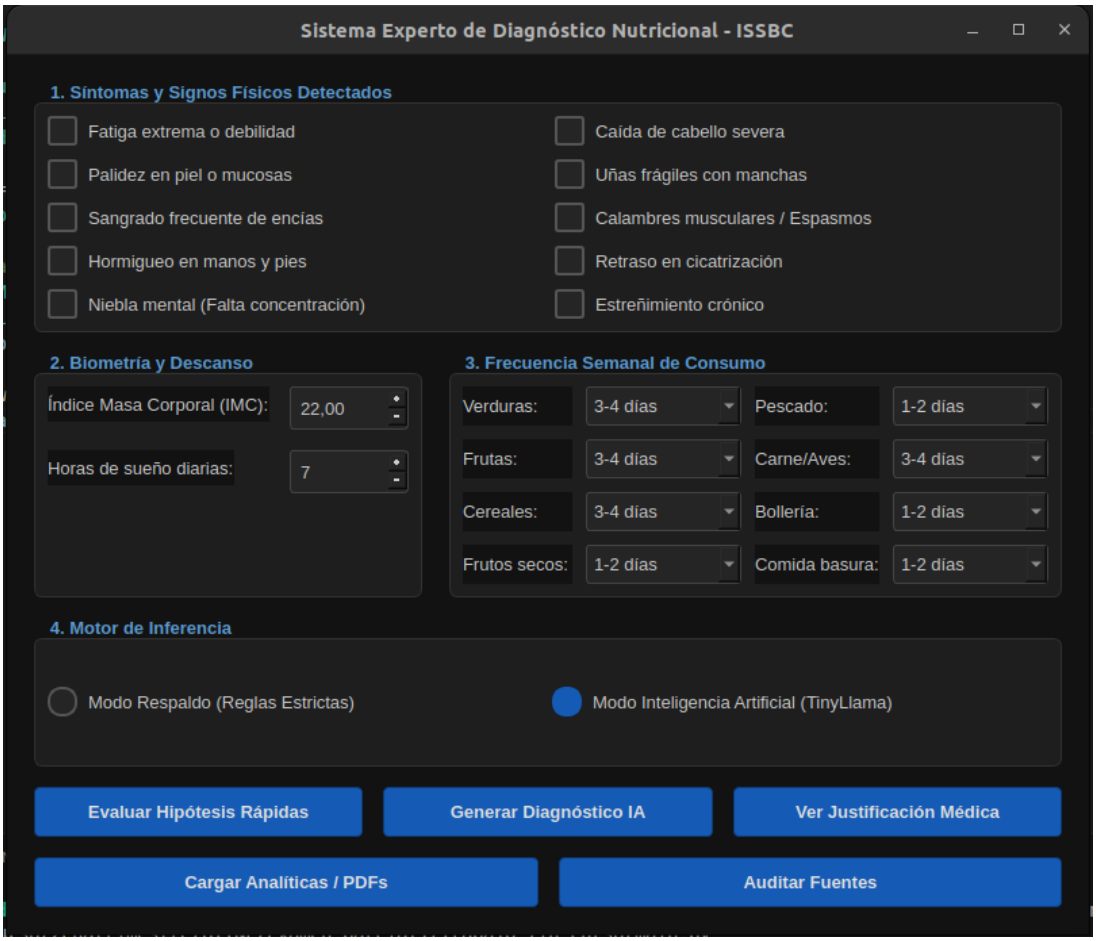

Figura 3: Captura de pantalla de la ventana principal mostrando los controles organizados.

5.3. Ventanas de diálogo secundarias

Para no sobrecargar la ventana principal con información emergente, he centralizado la creación de los cuadros de diálogo en un archivo independiente. He programado varias clases que heredan de la clase base de diálogos, cada una con un propósito específico dentro del flujo de la aplicación.

La primera de estas clases genera una ventana con una tabla de datos donde se exponen las hipótesis calculadas por el motor de inferencia local. Otra clase proporciona un cuadro de texto enriquecido de solo lectura, el cual utilizo para volcar en pantalla el informe generado por la inteligencia artificial y su correspondiente justificación médica. Finalmente, he implementado dos ventanas adicionales para la gestión de fuentes documentales.

```
# -*- coding: utf-8 -*-
from PyQt6.QtWidgets import (QDialog, QVBoxLayout, QHBoxLayout,
    QPushButton,
                             QTableWidget, QTableWidgetItem, QTextEdit,
                             QListWidget, QHeaderView)
```

```python

class DialogoHipotesis(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Hip tesis Posibles")
        self.resize(550, 300)
        layout = QVBoxLayout(self)
        self.tabla = QTableWidget(0, 3)
        self.tabla.setHorizontalHeaderLabels(["Hip tesis", "Probabilidad", "Estado"])
        self.tabla.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeMode.Stretch)
        layout.addWidget(self.tabla)
        btn_cerrar = QPushButton("Cerrar")
        btn_cerrar.clicked.connect(self.accept)
        layout.addWidget(btn_cerrar)

class DialogoTexto(QDialog):
    def __init__(self, titulo, parent=None):
        super().__init__(parent)
        self.setWindowTitle(titulo)
        self.resize(450, 350)
        layout = QVBoxLayout(self)
        self.texto = QTextEdit()
        self.texto.setReadOnly(True)
        layout.addWidget(self.texto)
        btn_cerrar = QPushButton("Cerrar")
        btn_cerrar.clicked.connect(self.accept)
        layout.addWidget(btn_cerrar)
```

Imagen 4: Estructura de las ventanas emergentes en dialogos_view.py

Figura 4: Vista de uno de los cuadros de diálogo secundarios creados para la aplicación.

6. Desarrollo del código: Controlador de Eventos

El controlador es la pieza fundamental que permite la comunicación bidireccional entre la interfaz gráfica y la lógica de negocio. En el archivo correspondiente, he implementado la clase principal del controlador que recibe como parámetros las instancias del modelo y de la vista durante su inicialización.

El primer paso en la construcción del controlador ha sido la definición de un método dedicado exclusivamente a conectar las señales emitidas por los componentes visuales con sus respectivas funciones de respuesta. De este modo, cuando el usuario interactúa con los botones de la interfaz, el controlador captura el evento y ejecuta la acción pertinente.

```
    def conectar_senales(self):
        self.vista.btn_evaluar.clicked.connect(self.procesar_hipotesis)
        self.vista.btn_diagnosticar.clicked.connect(lambda: self.
            iniciar_procesamiento_ia("diagnostico"))
        self.vista.btn_justificacion.clicked.connect(lambda: self.
            iniciar_procesamiento_ia("justificacion"))
        self.vista.btn_pdfs.clicked.connect(self.abrir_gestion_pdfs)
        self.vista.btn_web.clicked.connect(self.abrir_fuentes_web)
        self.vista.radio_web.toggled.connect(self.cambiar_modo)
```

Imagen 5: Conexión de señales y eventos en main_controller.py

Para garantizar que el modelo trabaje siempre con la información más reciente introducida por el facultativo, he desarrollado un método de actualización de datos. Esta función extrae el estado de las casillas de verificación de los síntomas, los valores numéricos de la biometría y las selecciones de los menús desplegables de la dieta, enviándolos al modelo justo antes de solicitar cualquier tipo de evaluación clínica.

```
    def actualizar_datos_basicos(self):
        sintomas = self.vista.obtener_sintomas_marcados()
        imc = self.vista.spin_imc.value()
        sueno = self.vista.spin_sueno.value()

        dieta = {
            "verduras": self.vista.combo_verduras.currentText(),
            "frutas": self.vista.combo_frutas.currentText(),
            "cereales": self.vista.combo_cereales.currentText(),
            "frutos_secos": self.vista.combo_frutos_secos.currentText(),
            "pescado": self.vista.combo_pescado.currentText(),
            "carne": self.vista.combo_carne.currentText(),
            "bolleria": self.vista.combo_bolleria.currentText(),
            "basura": self.vista.combo_basura.currentText()
        }
        self.modelo.actualizar_datos_paciente(sintomas, imc, sueno,
            dieta)
```

Imagen 6: Extracción y actualización de datos desde la vista al modelo

Finalmente, he programado las funciones encargadas de invocar el motor de inferencia. Si el usuario solicita el análisis rápido de respaldo, el controlador instruye al modelo para que aplique las reglas locales y posteriormente vuelca los resultados en la ventana de diálogo correspondiente. En el caso de requerir el análisis avanzado por inteligencia artificial, el controlador delega la tarea en el hilo de ejecución secundario y actualiza el

estado de los botones para informar al usuario de que el sistema se encuentra procesando la petición sin congelar la ventana.

7. Conclusiones

El desarrollo de este sistema experto ha supuesto la consolidación de los conocimientos teóricos y prácticos adquiridos durante la asignatura. La implementación estricta del patrón Modelo-Vista-Controlador ha permitido integrar con éxito una interfaz gráfica profesional, un sistema local de reglas lógicas y un modelo de lenguaje de inteligencia artificial. El resultado final cumple con los requisitos clínicos de usabilidad, robustez tecnológica y tolerancia a fallos mediante su sistema dual de diagnóstico. Todo el control de versiones y el despliegue se han gestionado de forma colaborativa, superando las limitaciones físicas del hardware disponible.

Referencias

[1] ZetCode. *PyQt5 tutorial - First programs.*
Enlace: https://zetcode.com/gui/pyqt5/firstprograms/.

[2] ZetCode. *PyQt5 tutorial - Menus and toolbars.*
Enlace: https://zetcode.com/gui/pyqt5/menustoolbars/.

[3] ZetCode. *PyQt5 tutorial - Layout management.*
Enlace: https://zetcode.com/gui/pyqt5/layout/.

[4] ZetCode. *PyQt5 tutorial - Events and signals.*
Enlace: https://zetcode.com/gui/pyqt5/eventssignals/.

[5] Riverbank Computing. *PyQt5 Reference Guide.*
Enlace: https://www.riverbankcomputing.com/static/Docs/PyQt5/.

[6] Google Gemini.
Enlace: https://gemini.google.com/app.